

GoRules ZEN Engine Guide

This repository uses GoRules.ZenEngine as the rule execution runtime. The package is an open-source, MIT-licensed .NET binding over the GoRules ZEN Engine core.

The important idea is this:

- your application owns the rule data and the input payload
- GoRules.ZenEngine evaluates a JSON decision model against that input
- the engine returns the decision result, which your app can display, store, or forward

This repo demonstrates that pattern with a small ASP.NET Core API and a React UI.

What the DLL is doing

In the API, the engine is used from GoRules.Api/Services/GoRulesDecisionService.cs.

The core runtime pattern is:

```
using var engine = new ZenEngine(loader: null, customNode: null);  
var decision = engine.CreateDecision(new JsonBufferType(Encoding.UTF8.GetByt  
var context = new JsonBufferType(Encoding.UTF8.GetBytes(JsonSerializer.Seria  
var response = await decision.Evaluate(context, null);
```

So the engine expects two things:

1. A decision model in JSON form.
2. A context payload in JSON form.

The engine does not require a React UI. The UI in this repo is only a way for a user to create rules and submit input.

What request it expects

The engine itself is not called directly from the browser. The app calls its own API, and the API builds the engine request.

Single rule execution

`EvaluateSingleRuleAsync` builds a decision model for one rule using:

- the rule name
- the rule conditions
- the rule result
- the input fields referenced by those conditions

It then sends the runtime input JSON to the engine.

Selected rule batch execution

`EvaluateSelectedRulesBatchAsync` builds one combined decision model from the selected rules and evaluates that model against the input JSON.

Does it need rules and request payload every time?

Yes, but in a specific sense.

The engine always needs:

- a decision model that represents the rules you want to run
- a context payload containing the values to test against those rules

That does **not** mean you must send your entire rules database on every call.

In this repo:

- a single-rule request sends only the selected rule plus the input JSON
- a multi-rule request sends only the selected rules plus the input JSON

The application decides which rules to include. The engine just evaluates the decision model it receives.

Can you run a single rule alone?

Yes.

This repo supports a single rule path through `/api/rules/execute` when `ruleId` is provided. The API loads that one rule, converts it into a decision model, and evaluates it.

That is the cleanest option when you already know which rule applies.

Can you run a combination of rules?

Yes.

This repo also supports selected-rule batch execution through `/api/rules/execute-all`.

That path is useful when:

- you want a first-match evaluation across multiple rules
- you want a combined decision model rather than separate calls
- you want to see which rule matched from the response

Can you identify which rule passed or failed from the response?

Yes.

The API response includes:

- `decision`
- `matchedRule`
- `message`
- execution time
- the original input
- the generated decision JSON
- the engine result JSON

For batch execution, the response also includes per-rule results so the UI can show which rule matched and which one failed.

What this repo sends to the engine

The repo does not send a human-written text prompt. It sends a JSON decision model.

Single rule model

For one rule, the engine receives a model with:

- an input node
- a decision table node

- an output node
- one row for the rule result
- one fallback row for no match

Combined model

For selected rules, the engine receives a combined decision table built from all selected rules.

That combined model is created from all operands used by those rules, so the table can compare the right fields and return the first match.

What the input JSON should look like

The input JSON should contain the fields referenced by the decision model.

Example:

```
{
  "PD_Doc_Borr1_Name_Last": "TOLDEO, ",
  "PD_Doc_Borr1_Name_First": "MARVIN"
}
```

If the rule compares those fields, the engine needs those values in the context payload.

Do you need to send all fields in a real system?

No.

The engine only evaluates the fields used by the decision model.

If a rule depends on 4 fields, the model only needs those 4 fields in the request context. You may still choose to send a larger payload if that is convenient for your application, but the engine itself is not requiring every field in your domain model.

What about 5000+ fields and 200 rules?

The short answer is: you should not design it as "send everything, every time" unless that is truly the simplest integration.

Better options are:

- split rules into decision models by business scenario

- load only the rules that apply to the current transaction
- keep the context payload focused on the data that the selected model needs
- pre-load or precompile models if you evaluate them frequently

If your application has 5000 fields, the engine does not automatically require all 5000 for every rule run.

What matters is:

- which fields the active rules reference
- how you group those rules
- how much data you choose to include in the context JSON

If you make one giant combined model with 200 rules and 5000 potential fields, you can still run it, but that is usually a design choice rather than a requirement.

In practice, it is often better to:

1. group related rules into smaller models
2. evaluate only the relevant group
3. keep the rule input schema as small as possible for that scenario

How this is different from handwritten code

If you write the logic yourself, it often looks like this:

```
if (borrowerFirstName == "MARVIN" && borrowerLastName == "TOLDEO")
{
    return "Approve";
}
```

That is fine for a few rules.

The tradeoff is that handwritten logic becomes harder to manage when rules are:

- numerous
- frequently changed
- reviewed by non-developers
- reused across products or customers
- audited or versioned separately from code releases

GoRules.ZenEngine helps when you want the rule definition to be data, not code.

Why use GoRules ZEN Engine

The main benefits are:

- rules live as data and can be versioned separately from application code
- the same rule model can be evaluated consistently across platforms
- rule changes do not always require a redeploy
- tracing helps explain why a decision happened
- the engine can be embedded inside your application rather than requiring a separate rules server
- rules are easier to inspect, store, and manage as JSON models

When you should use it

Use it when:

- business policy changes often
- business users or analysts need visibility into rule definitions
- you need auditability or traceability
- the same decision logic must be reused in multiple services
- you want to separate rule maintenance from application code

When you may not need it

You may not need a rules engine if:

- the logic is tiny and changes rarely
- the conditions are simple and local to one service
- nobody needs to edit or inspect the rules outside the codebase

In that case, plain code can be simpler.

Do you still need your own UI?

Usually, yes.

GoRules.ZenEngine is the execution engine. It does not give you a full application UI for your business users.

In a real product, you typically build:

- a rule authoring UI
- a rule review workflow
- storage for rules and versions
- an execution API
- optionally, an audit and trace view

This repo includes a small React UI that plays that role.

How this repo is structured

- GoRules.Api/Program.cs exposes REST endpoints for rules and execution
- GoRules.Api/Services/GoRulesDecisionService.cs builds decision JSON and calls the engine
- GoRules.React/src/App.jsx provides the UI
- GoRules.Api/Data/SeedData.cs seeds example rules

Tracing and debugging

The package supports tracing.

That is useful when you need to answer questions like:

- which node ran
- which input matched
- why a rule was rejected
- which branch was taken

For production-style troubleshooting, tracing is one of the main advantages over a hand-written `if/else` chain.

Open source and license

According to the package documentation, GoRules ZEN Engine is open source and MIT licensed.

The broader GoRules platform includes additional offerings, but the engine package itself is open source.

Practical recommendation for this repo

For this project, a good architecture is:

1. store rules in the API/database
2. let the UI manage rule authoring
3. build decision JSON in the API
4. send only the relevant input JSON to the engine
5. use single-rule execution when one rule is selected
6. use combined execution when you need first-match evaluation across multiple rules
7. keep rule groups small enough to be understandable and maintainable

That is the real value of the engine: you keep rule logic out of application code, while still embedding the runtime in your own app.